

Team: Suchintan Biswas (24BCE5237), Shubham Pappu Tiwari (24BCE1079) · **Faculty:** Dr. Suganeshwari G · **Slot:** B2+TB2

Languages: C · C++ · Java · Python · JavaScript · TypeScript · **Stack:** React/TS · FastAPI · PostgreSQL · Redis · SQLAlchemy · Google Gemini · Docker · JWT

1. PROBLEM STATEMENT

Existing code-review approaches typically rely on either narrow deterministic rule coverage or opaque, context-dependent AI-only findings and scoring. **CodePilot combines deterministic and AI-assisted analysis in one platform to produce explainable, reproducible quality scores.**

2. DATASET DESCRIPTION

Not a conventional supervised ML system; no training dataset is used. Evaluation relies on independently defined validation sets.

- Source:** project-created controlled Python benchmark plus independently defined validation/holdout sets; ground truth fixed independently of CodePilot output.

Size:	Primary benchmark	20 files (4 each: clean / low / medium / high / critical)
	Model-property cases	12 cases
	Unseen holdout	15 files (never used to tune the model)

- Features:** LOC, cyclomatic complexity, severity, confidence, security / performance / semantic defects, maintainability characteristics.
- Preprocessing:** ingestion → language/syntax validation → LOC & complexity extraction → finding normalization → confidence assignment → deterministic scoring.

3. METHODOLOGY / ARCHITECTURE

Approach. A hybrid pipeline: deterministic analysis and Gemini AI *find* issues, and a separate, deterministic scoring model (not ML) *converts* them into scores.

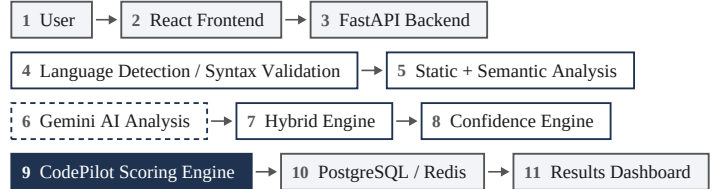


Fig. 1. System pipeline. Dashed box: advisory AI stage that does not set the final score. Filled box: deterministic scoring.

- Static + Semantic analysis:** deterministic rules for common defect patterns, plus deeper language-specific rules across all six languages.
- Gemini analysis:** complementary contextual reasoning; it does not set the final score.
- Hybrid Engine:** merges and deduplicates deterministic and AI findings.
- Scoring Engine:** independently converts findings and code metrics into deterministic, bounded, explainable scores.

SCORING ENGINE VALIDATION

Custom deterministic formulas grounded in established software-quality concepts (not ISO / CVSS / SQALE / Radon equations).

Impact = SeverityWeight × Confidence/100
weights: Critical 10 · High 7 · Medium 4 · Low 1
NormImpact = TotalImpact ÷ LOC_K
IssueDensity = Σ(Confidence/100) ÷ LOC_K
AvgComplexity = TotalCyclomatic ÷ #Functions
TechDebtHealth = clamp(100 - 2·NormImpact, 0, 100)

Maintainability = clamp(100 - [1.5·NormImpact + IssueDensity + AvgComplexity/2], 0, 100)
Security = clamp(100 - 3·SecImpact, 0, 100)
Performance = clamp(100 - 3·PerfImpact, 0, 100)
Overall = 0.30·Security + 0.25·Performance + 0.30·Maintainability + 0.15·TechDebtHealth

Mathematical validation. All six property checks pass: boundedness in [0, 100]; severity, confidence and complexity monotonicity; non-improvement when issues are added; and exact weighted aggregation (weights sum to 1.0).

4. RESULTS & INSIGHTS

Table 1. External empirical validation (Spearman ρ).

Dimension	Reference	n	ρ	p-value
Maintainability	Radon Maintainability Index	20	+0.3189	0.1707
Security	Independent benchmark severity	20	-0.3853	0.0935
Performance	Unseen holdout (15 files)	15	-0.7890	< 0.001*

* Significant at $p < 0.05$. Performance holdout: precision 100%, recall 62.5% (5 TP / 3 FN).

Key findings

- Strongest evidence is for Performance:** $\rho = -0.789$, $p < 0.001$ on an unseen holdout (revised metric).
- Security ($\rho = -0.385$, $p = 0.094$) and Maintainability ($\rho = +0.319$, $p = 0.171$) trend in the expected direction but remain statistically preliminary at $n = 20$.
- Mathematical properties (boundedness, monotonicity, exact aggregation) hold across all tested cases; 46 scoring-engine and 75 Python-analyzer tests pass.

Performance

$n = 15$

Security

$n = 20$

Maintainability

$n = 20$

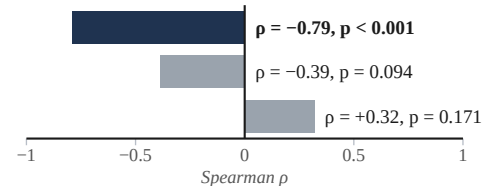


Fig. 2. Spearman ρ of CodePilot scores against each independent reference.

Sensitivity analysis: scores are stable, not brittle

- 27 Maintainability-coefficient and 6 Overall-weight configurations tested; rank correlation stays 0.9392–0.9989 (Maintainability) and 0.9936–0.9967 (Overall).
- Maximum score change 25.75 pts; maximum ranking reversals 39 / 190 pairs.
- This demonstrates parameter stability, not universal correctness of the coefficients.*

5. NOVELTY

The novelty of CodePilot is primarily in the integration of deterministic multi-language semantic analysis, complementary Gemini contextual reasoning, Hybrid Engine fusion, confidence-aware processing, and an independent explainable scoring engine into one six-language code-review platform. The individual techniques are established; **our contribution is their implementation and integration into one validated system.**

Conclusion. CodePilot is a multi-language hybrid code-review platform with a custom deterministic scoring model, grounded in established software-quality concepts, mathematically validated for boundedness and monotonicity, stress-tested for parameter stability, and empirically evaluated against independent references. The strongest out-of-sample evidence was observed for Performance ($\rho = -0.789$, $p < 0.001$), while Security and Maintainability evidence remains preliminary. The model is best characterized as a validated engineering metric for the project's scope.